Realtime One-to-One Chat App

A step-by-step learning roadmap for building a WhatsApp-style interface with React, Node.js, Express, MongoDB, and Socket.IO.

PREPARED FOR
Abhishek Ranjan

ROADMAP DATE
23 August 2026

CURRENT MILESTONE
Authentication sessions

LEARNING METHOD
You implement each task; the guide defines what and why.

Project promise: users create a unique username, optionally upload a profile picture, invite or add a friend, exchange private messages, and see a green online indicator when the friend is connected.

This roadmap intentionally describes responsibilities, contracts, checks, and documentation without giving you a complete copy-and-paste implementation.

How To Use This Guide

1. Work through the phases in order. Do not start Socket.IO before authentication and message persistence are reliable.
2. For each task, write the code yourself, test the expected behavior, and commit the working milestone.
3. When stuck, show the relevant file, exact request, response status/body, and terminal error. Ask for a review of your attempt.
4. Use the linked official documentation before relying on tutorials. Tutorials age quickly; contracts in official docs are the source of truth.

Contents

1. Current Project Audit

The server already has a useful authentication foundation. The audit is based on the current project files in `Realtime chat app/server`.

- [x] Express server and MongoDB connection exist.
- [x] The User model stores name, unique normalized username, hidden password hash, profile picture, last seen, and timestamps.
- [x] Registration validates input, checks username availability, hashes the password, and creates the user.
- [x] A separate username-availability endpoint supports live feedback on the future registration screen.
- [x] Login finds the hidden password hash and checks the submitted password.
- [x] Registration, username checks, and credential login have been manually tested.
- [ ] JWT session creation and cookie handling are not yet connected.
- [ ] `generateToken.js` exists but still needs its responsibility implemented.
- [ ] Logout, current-user, and authentication middleware are still required.
- [ ] The React client is not yet initialized into the working application.

> **Your immediate next milestone:** finish cookie-based authentication. A successful password comparison proves identity once; the browser still needs a secure session for later HTTP requests and the Socket.IO connection.

2. Target Architecture

REST should handle account actions, friend workflows, conversation lists, and paginated message history. **Socket.IO** should handle live messages, typing, read status, and presence. **MongoDB** remains the durable source of truth: persist a message before broadcasting it.

> **Important distinction:** online status is temporary connection state; `lastSeen` is durable historical data. Do not create one database write for every heartbeat. Update `lastSeen` when the user's final active socket disconnects.

3. Remaining Build Phases

Phase 1 - Complete Authentication Sessions DO THIS NOW

Goal: after login or registration, the browser remains authenticated without receiving the password hash.

1. In `server/utils/generateToken.js`, define one small responsibility: sign a JWT containing the user's ID. Read the secret and expiry from environment variables.
2. Configure `cookie-parser` in `server.js` so later middleware can read the session cookie.
3. After successful registration and login, create the token and place it in an `HttpOnly` cookie. Use `SameSite`, `Secure` in production, a limited lifetime, and a consistent cookie name.
4. Create `server/middleware/authenticate.js`. It reads the cookie, verifies the token, loads the user without `passwordHash`, and either attaches the user to the request or returns `401`.
5. Add `GET /api/auth/me` to return the currently authenticated public user. This restores login after a page refresh.
6. Add `POST /api/auth/logout` to clear the cookie using matching cookie options.
7. Test missing, invalid, expired, and valid cookies. Also confirm the cookie is HttpOnly in browser developer tools.

Completion check: register/login sets a cookie; `/me` succeeds with it and returns `401` without it; logout makes the same cookie unusable.

Phase 2 - Initialize and Connect the React Client NEXT

Goal: create working register, login, and authenticated app screens backed by the API.

1. Initialize Vite with React inside `client`. Learn the generated files before deleting or reorganizing them.
2. Add server CORS configuration with the exact client origin and credentials enabled. On the client, fetch session endpoints with credentials included.
3. Create feature folders for `auth`, `chat`, reusable `components`, and API utilities. Keep network code out of visual components where practical.
4. Create an authentication provider that calls `/me` once on startup and represents loading, authenticated, and anonymous states.
5. Build registration and login forms. Show field-specific server errors and disable duplicate submissions.
6. Call the username-check endpoint after a short debounce, not on every keystroke. The register endpoint must still perform the final uniqueness check.
7. Protect the chat screen from anonymous users and redirect authenticated users away from login/register screens.

Completion check: a user can register, refresh the browser without losing the session, log out, and log back in.

Phase 3 - Profile Picture Upload AFTER AUTH UI

Goal: safely upload or replace an avatar and store only useful image metadata in MongoDB.

1. Create a protected profile route and controller. Keep upload handling separate from registration until the basic account flow is stable.
2. Use multipart middleware such as Multer. Enforce image MIME types and a small size limit before sending data to an image host.
3. Upload from the server to Cloudinary or another image service. Store the returned secure URL and public asset ID on the User document.
4. When replacing an avatar, remove the prior hosted asset after the new upload succeeds. Provide a default avatar when no picture exists.

Completion check: invalid or oversized files are rejected; valid avatars survive refresh and appear everywhere the user is shown.

Phase 4 - Friend and Invitation Workflow

Goal: users can find another unique username and explicitly establish permission to chat.

1. Create `FriendRequest` with sender, receiver, status, and timestamps. Add indexes that prevent duplicate active requests.
2. Create protected endpoints to search by username, send, list, accept, reject, and cancel requests.
3. Reject self-invites, duplicate requests, already-friends cases, and operations by someone other than the sender/receiver.
4. Choose how accepted friendship is stored: a separate Friendship document is easiest to query and extend for blocking.
5. Only after the basic request flow works, consider shareable invite links with random, expiring, one-time tokens.

Completion check: two accounts can become friends, while unrelated users cannot open a private conversation.

Phase 5 - Conversations and Stored Messages

Goal: build correct private-message persistence before adding realtime delivery.

1. Create a Conversation model for exactly two participants. Use a canonical participant key or another unique strategy so the same pair cannot create duplicate conversations.
2. Create a Message model containing conversation, sender, text, delivery/read state, optional client message ID, and timestamps.
3. Add protected REST endpoints to start/open a conversation, list conversations, and fetch older messages with cursor-based pagination.
4. For every conversation and message operation, verify that the authenticated user is a participant.
5. Limit text length and reject empty or whitespace-only messages. Return public sender data only.

Completion check: two friends can send and reload stored messages using REST alone; a third account receives `403` or `404`.

Phase 6 - Socket.IO Realtime Messaging

Goal: deliver persisted messages immediately and safely across tabs and devices.

1. Create an HTTP server from Express and attach Socket.IO to that same server.
2. Authenticate every socket during the connection handshake. Never trust a user ID sent by the client as proof of identity.
3. Join a private room based on the authenticated user ID. Track `userId -> Set(socketIds)` because one user may have multiple tabs or devices.
4. For send-message: validate, confirm membership, persist, update conversation summary, then emit the saved message to participant rooms.
5. Use acknowledgements so the sender knows whether a message was saved. A client-generated unique ID prevents duplicates after retries.
6. Add typing and read events only after live send/receive is reliable.

Completion check: messages appear instantly in two browsers, persist after reload, and do not duplicate during reconnect/retry.

Phase 7 - Online Green Dot and Last Seen

Goal: accurately show whether a friend has at least one live authenticated connection.

1. Mark a user online when the first authenticated socket connects. Mark offline only after the final socket disconnects.
2. Send presence only to friends or conversation participants, not to every connected account.
3. When the final socket closes, save `lastSeen` and emit the offline timestamp.
4. On initial chat-list load, combine stored last-seen data with the server's current presence state.
5. Test closing one of two tabs, temporary network loss, reconnect, logout, and server restart.

Completion check: the green dot remains while any tab is connected and disappears promptly after the last connection closes.

Phase 8 - WhatsApp-Style Chat Interface

Goal: build a responsive, work-focused chat experience rather than only a visual imitation.

1. Desktop: chat list/sidebar on the left and active conversation on the right. Mobile: navigate between list and conversation.
2. Chat rows show avatar, name, last-message preview/time, unread count, and the presence dot on the avatar.
3. The conversation shows a header, paginated message history, delivery/read state, typing state, and a stable composer.
4. Implement loading, empty, offline, retry, and error states. Use temporary message IDs for optimistic UI and visibly fail messages the server rejects.
5. Preserve scroll position when older messages load and auto-scroll only when the user is already near the newest message.

Phase 9 - Hardening, Automated Tests, and Deployment

Goal: make the app reliable outside your local machine.

1. Add central error handling, consistent response shapes, request validation, authentication rate limits, secure headers, payload limits, and production CORS/cookie settings.
2. Add unit tests for validation/helpers, API integration tests, Socket.IO integration tests, and critical frontend flow tests.
3. Deploy MongoDB, the Node service on a host supporting WebSockets, and the built React client. Configure HTTPS and environment secrets.
4. Add a health endpoint, useful logs without secrets, database backups, and production smoke tests with two accounts.

4. Data Model Plan

Collection	Important fields	Rules and indexes
User	`name`, `username`, `passwordHash`, avatar URL/public ID, `lastSeen`, timestamps	Unique normalized username. Hide password hash by default. Never return it in API responses.
FriendRequest	`sender`, `receiver`, `status`, timestamps	No self-request. Prevent duplicate pending pairs. Only receiver accepts/rejects.
Friendship	canonical user pair, created timestamp	One unique document per pair. Use it to authorize chat and presence visibility.
Conversation	two participants, canonical pair key, last message reference/preview/time, timestamps	One private conversation per pair. Index participant queries and recent activity.
Message	`conversation`, `sender`, `text`, `clientMessageId`, delivered/read timestamps, timestamps	Index conversation plus creation time. Unique sender/client ID for retry deduplication.
Invite (optional)	creator, hashed random token, expiry, used timestamp	Short lifetime, one-time use, never store a sensitive raw token when a hash will work.

> **Modeling rule:** keep the password hash with the User document. A separate password collection adds joins and failure cases without improving protection. Security comes from strong hashing, hidden selection, access control, secrets management, and HTTPS.

5. REST API Plan

Method and path	Purpose	Status
`POST /api/auth/register`	Create a user after server-side validation and uniqueness check.	Built
`GET /api/auth/check-username`	Give the form early availability feedback; never replace the register check.	Built
`POST /api/auth/login`	Verify credentials and establish a cookie session.	Credentials built; session next
`POST /api/auth/logout`	Clear the session cookie.	Next
`GET /api/auth/me`	Restore the authenticated user after refresh.	Next
`PATCH /api/users/me/avatar`	Upload/replace the current user's profile picture.	Planned
`GET /api/users/search?username=...`	Find users without exposing private fields.	Planned
`POST /api/friend-requests`	Send a friend request.	Planned
`GET /api/friend-requests`	List received/sent pending requests.	Planned
`PATCH /api/friend-requests/:id`	Accept or reject with authorization checks.	Planned
`GET /api/conversations`	Return the chat list with summaries and unread information.	Planned
`POST /api/conversations`	Open or return the existing private conversation with a friend.	Planned
`GET /api/conversations/:id/messages`	Return an authorized cursor-paginated message page.	Planned

Exact paths can follow your existing naming style. Consistency matters more than copying these names literally.

6. Socket Event Contract

Event	Direction	Responsibility
`message:send`	Client -> server	Contains conversation, text, and client message ID. Server validates, authorizes, saves, and acknowledges.
`message:created`	Server -> participants	Broadcast the saved public message object, including its database ID and timestamps.
`message:read`	Client -> server -> sender	Authorize the reader, update durable state, and notify the other participant.
`typing:start` / `typing:stop`	Client -> server -> peer	Temporary state; rate-limit and never write each event to MongoDB.
`presence:changed`	Server -> permitted friends	Send user ID, online boolean, and last-seen timestamp when offline.
`connect_error`	Server/client lifecycle	Handle invalid or expired authentication without an infinite reconnect loop.

7. Security and Reliability Checklist

- [ ] Keep JWT secret, database URI, and image-service secret in environment variables; commit only an example env file.
- [ ] Use HttpOnly cookies, HTTPS, Secure in production, appropriate SameSite, and a deliberate CSRF strategy.
- [ ] Validate every REST body, query, route parameter, upload, and socket payload on the server.
- [ ] Authorize membership for every conversation read/write and every socket event.
- [ ] Apply rate limits to registration, login, username search, invitations, uploads, and message bursts.
- [ ] Use Helmet, narrow CORS origins, JSON body limits, upload limits, and maximum message length.
- [ ] Never log passwords, password hashes, session tokens, raw invite tokens, or private message bodies in routine logs.
- [ ] Use cursor pagination and database indexes; never return an unlimited message history.
- [ ] Use acknowledgements and unique client message IDs so reconnect/retry does not create duplicates.
- [ ] Treat end-to-end encryption as a separate advanced project. Do not market ordinary TLS/database storage as E2EE.

8. Testing Strategy

Level	Minimum cases
Unit	Username normalization/pattern, empty-space rejection, token helper, validation schemas, canonical pair calculation.
API integration	Duplicate username race, bad password, cookie lifecycle, protected endpoints, self/duplicate friend request, unauthorized conversation access, pagination.
Socket integration	Unauthenticated connection, two-user delivery, third-user denial, multiple tabs, disconnect/reconnect, acknowledgement, duplicate client ID.
Frontend	Debounced username result, register/login errors, refresh session, logout, optimistic message success/failure, mobile navigation, presence update.
Manual production smoke test	Two fresh accounts in separate browsers: add friend, upload avatar, chat live, close all tabs, verify green dot and last seen.

9. Deployment Checklist

1. Create a production MongoDB database and a least-privilege application user. Restrict network access appropriately.
2. Deploy Node/Express to a service with persistent WebSocket support. Confirm any proxy timeout and CORS configuration.
3. Build and deploy the React client. Point its API/socket base URL to the production Node service.
4. Set production environment variables: database URI, JWT secret, allowed client origin, runtime mode, image credentials, and cookie settings.
5. Use HTTPS everywhere. Verify Secure cookies are sent, cross-origin credentials work, and no secret appears in the browser bundle.
6. Add a health endpoint, error monitoring/log retention, database backups, and a rollback plan.

10. Learning Checklist

Learn	You should be able to explain
HTTP and Express	Method/path, status codes, headers, middleware order, cookies, CORS, controller responsibility.
MongoDB and Mongoose	Schema vs document, ObjectId references, unique indexes, validation, selection, indexes, pagination, population.
Authentication	Hash vs encryption, JWT signing/verification, cookie flags, authentication vs authorization.
React	Components, props, state, controlled forms, effects, context, routing, loading/error states.
Realtime systems	Socket lifecycle, rooms, acknowledgements, reconnect, multiple devices, ephemeral vs durable state.
Testing	Arrange/act/assert, isolated unit tests, integration boundaries, deterministic cleanup, negative authorization tests.
Deployment	Environment separation, HTTPS, build output, WebSocket hosting, logs, backups, health checks.

11. Definition of Done

- [ ] A username is normalized and guaranteed unique by a database index.
- [ ] Passwords are securely hashed and never returned or logged.
- [ ] Session survives refresh and ends on logout/expiry.
- [ ] Users can upload and replace a validated profile picture.
- [ ] Only accepted friends can create/open their one-to-one conversation.
- [ ] Messages persist, paginate, arrive live, acknowledge failures, and avoid retry duplicates.
- [ ] Presence works correctly with multiple tabs and updates last seen on final disconnect.
- [ ] Unauthorized users cannot read, send, mark read, or subscribe to another conversation.
- [ ] The UI works on desktop and mobile with loading, empty, error, and offline states.
- [ ] Critical API/socket tests pass and the deployed two-browser smoke test succeeds.

12. Official Documentation Links

Foundation and Backend

- Node.js download and Node test runner
- Express routing, middleware, and official middleware directory
- Express production security practices
- Mongoose schemas and indexes, connections, and populate
- Mongoose Model.exists() and MongoDB unique indexes
- MongoDB Node.js driver getting started

Authentication and Browser Security

- bcrypt for Node.js
- jsonwebtoken documentation
- MDN Set-Cookie reference and secure cookie configuration
- OWASP Password Storage Cheat Sheet

React and Client Requests

- React Learn / Quick Start
- React useEffect reference and Synchronizing with Effects
- Vite Getting Started and production build
- MDN Using Fetch and Request credentials

Realtime Messaging

- Socket.IO tutorial and emitting events
- Socket.IO rooms, middleware, and client authentication options
- OWASP WebSocket Security Cheat Sheet

Uploads, Testing, and Deployment

- Cloudinary Node integration and Node image/video upload
- SuperTest HTTP assertions and Vitest guide
- Vite static deployment guide
- Helmet security headers

Start Here: Your Very Next Task

> **Implement and understand JWT creation in `server/utils/generateToken.js`.**
>
> Before writing it, read the jsonwebtoken documentation sections for `jwt.sign()` and `jwt.verify()`. Decide:
>
> 1. What single user identifier belongs in the payload?
> 2. Which environment variable stores the signing secret?
> 3. How long should the login session last?
> 4. Should this helper only return the token, or also set the HTTP cookie? Prefer one clear responsibility.
>
> Then test that the helper returns a token and that verification recovers the expected user ID. Do not put a password, password hash, full user document, or secret data inside the JWT payload.

Roadmap version 1.0. Update the completed checklist after each verified milestone. Dependencies and hosting products change, so consult the linked official documentation when implementing each phase.